PYTHON CASE STUDY REPORT

NAME: VELIVENTI LASYAPRIYA

BRANCH: CSE

SECTION: 01

ROLL NUMBER:26000069

CASE STUDY 27: ADVANCED NUMERIC PATTERN GENERATOR

1. Problem Statement

Develop a Python program to generate different numeric patterns based on the number of rows entered by the user. The program should display an increasing number pattern in which each row contains numbers from 1 up to the row number.

2. Objectives

- To understand nested loops in Python.
- To generate numeric patterns using loops.
- To accept input from the user.
- To improve logical thinking and programming skills.

3. Assumptions

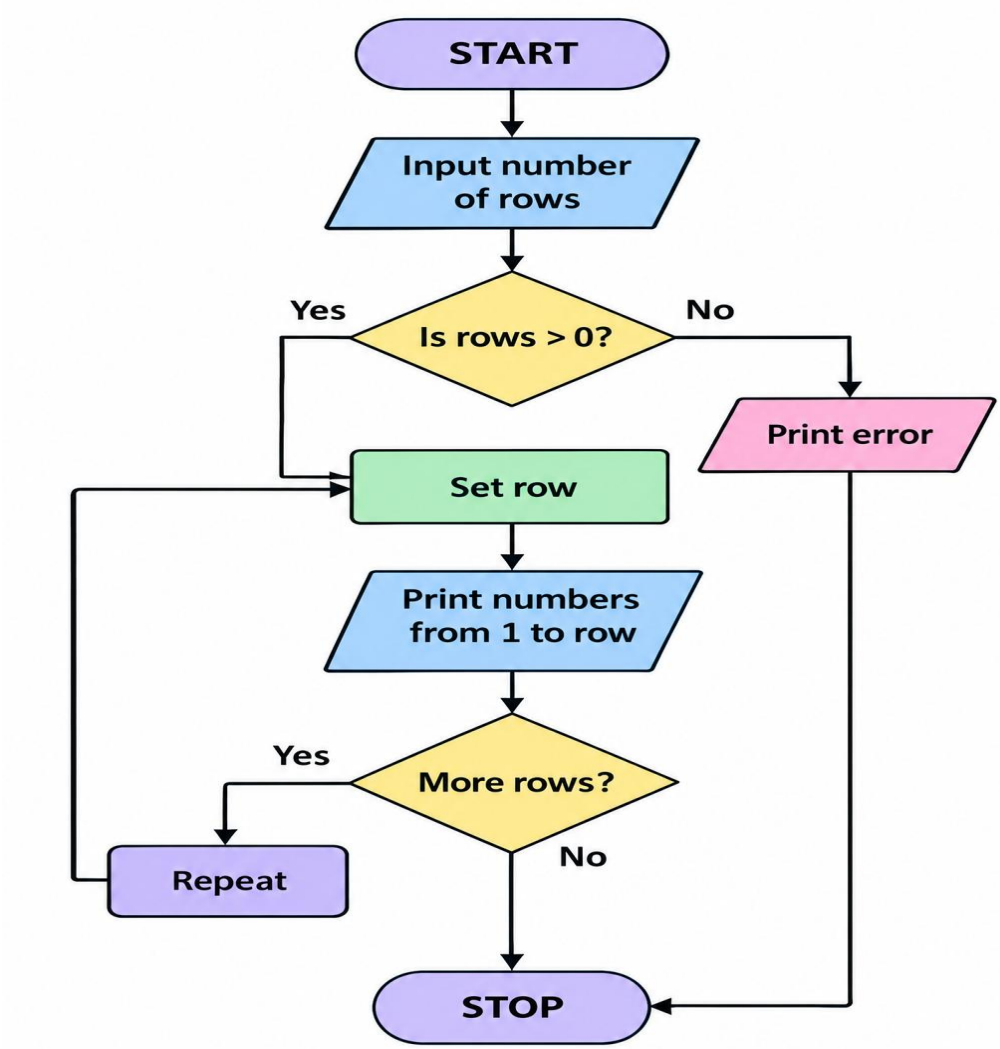
- The number of rows must be a positive integer.
- The program generates an increasing numeric triangle.
- Each row begins with 1.
- The number of elements in each row equals its row number.

4. Algorithm

1. Start the program.
2. Read the number of rows from the user.
3. Check whether the number of rows is positive.
4. Use an outer loop to control the rows.
5. Use an inner loop to print numbers from 1 to the current row number.
6. Move to the next line after printing each row.
7. Display an error message if the input is invalid.

8. Stop the program.

5.Flowchart



6. Pseudocode

```
BEGIN
INPUT rows
  IF rows <= 0 THEN
    PRINT "Enter a positive number"
  ELSE
    FOR i FROM 1 TO rows
```

```
        FOR j FROM 1 TO i
          PRINT j
        END FOR
        MOVE TO NEXT LINE
      END FOR
  END IF
END
```

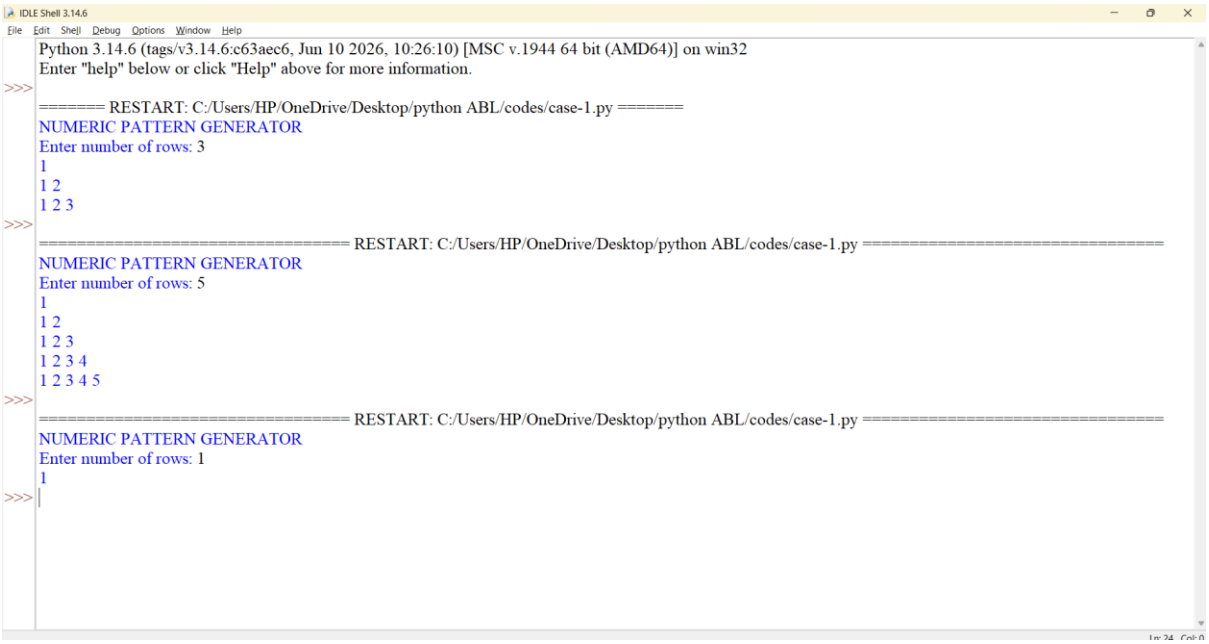
7. Python Source Code

```
print("NUMERIC PATTERN GENERATOR")
rows = int(input("Enter number of rows: "))
if rows <= 0:
   print("Please enter a positive number.")
else:
   for i in range(1, rows + 1):
      for j in range(1, i + 1):
         print(j, end=" ")
      print()
```

8. Test Cases

Test Case	Input	Expected Output
1	3	Three rows of numbers
2	5	Five rows of numbers
3	1	1

9.Screenshots



10. Conclusion

The Numeric Pattern Generator demonstrates how nested loops can be used to create numeric patterns. It helps develop an understanding of iteration, user input, and conditional statements in Python.

CASE STUDY 13: ATM TRANSACTION SEQUENCE SIMULATION

1. Problem Statement

Develop a Python program that simulates basic ATM transactions. The program should allow a user to check their balance, deposit money, and

withdraw money. It should update the balance after every successful transaction and prevent withdrawals that exceed the available balance.

2. Objectives

- To simulate basic ATM operations.
- To understand conditional statements and loops.
- To perform calculations involving money.
- To validate transactions.
- To update account balances correctly.

3. Assumptions

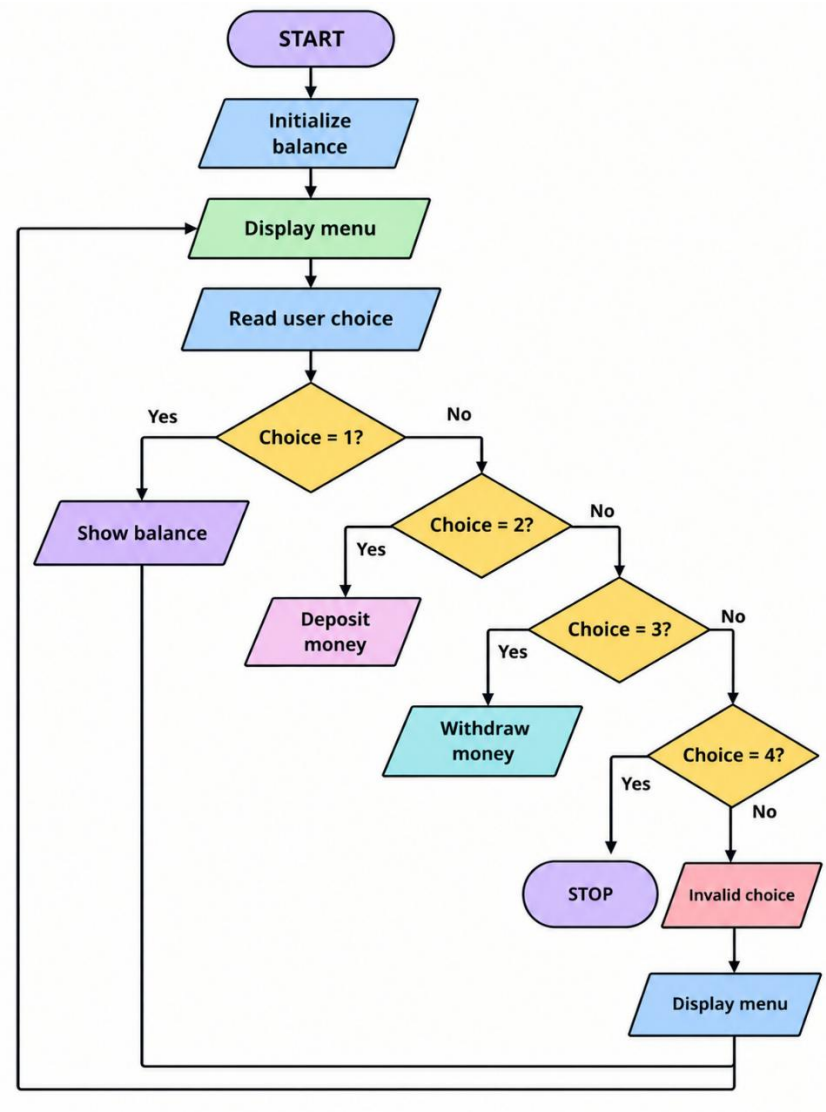
- The initial account balance is ₹5,000.
- The user can check their balance, deposit money, or withdraw money.
- Deposit and withdrawal amounts must be positive.
- Withdrawals cannot exceed the available balance.
- The program ends when the user selects Exit.

4. Algorithm

1. Start the program.
2. Initialize the balance to ₹5,000.
3. Display the ATM menu.
4. Ask the user to select an operation.
5. If the selection is 1, display the balance.
6. If the selection is 2, accept a deposit amount and update the balance.
7. If the selection is 3, accept a withdrawal amount.
8. Check whether the withdrawal amount is positive and does not exceed the balance.

9. If the selection is 4, terminate the program.
10. Display an error message for invalid menu choices.

5.Flowchart



6. Pseudocode

BEGIN

balance = 5000

REPEAT

```
DISPLAY menu
INPUT choice
IF choice = 1 THEN
    DISPLAY balance
ELSE IF choice = 2 THEN
    INPUT amount
    IF amount > 0 THEN
        balance = balance + amount
    ELSE
        DISPLAY invalid amount
    END IF
ELSE IF choice = 3 THEN
    INPUT amount
    IF amount <= 0 THEN
        DISPLAY invalid amount
    ELSE IF amount <= balance THEN
        balance = balance - amount
    ELSE
        DISPLAY insufficient balance
    END IF

ELSE IF choice = 4 THEN
    EXIT LOOP

ELSE
```

```
        DISPLAY invalid choice
      END IF

  UNTIL choice = 4
END
```

7. Python Source Code

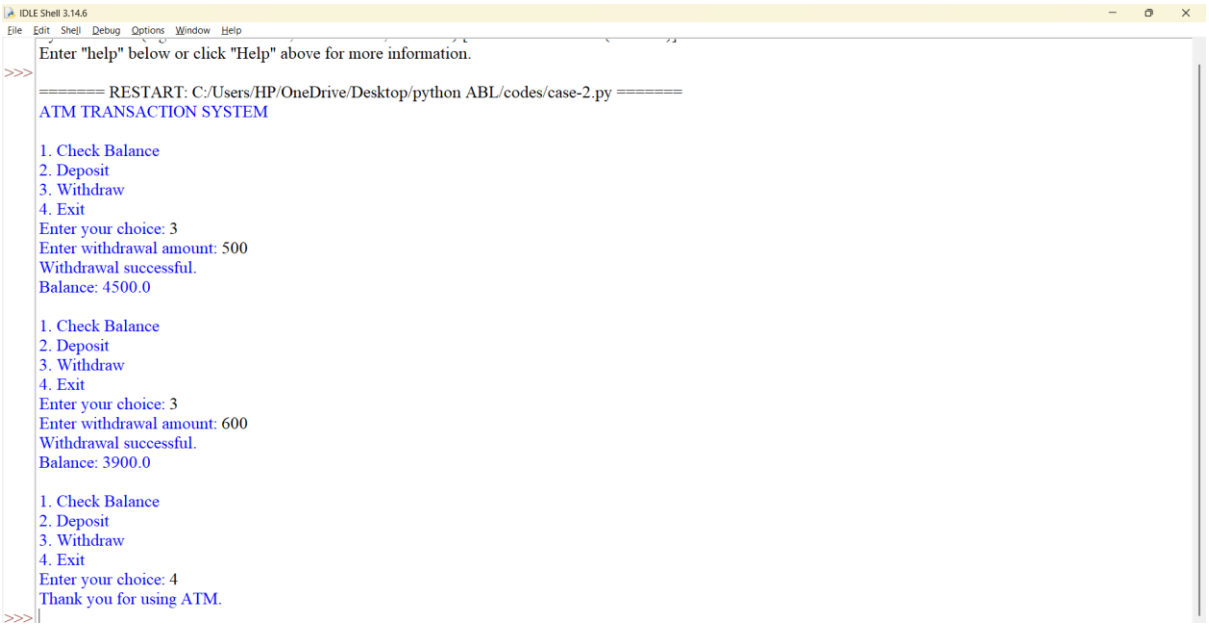
```
print("ATM TRANSACTION SYSTEM")
balance = 5000
while True:
   print("\n1. Check Balance")
   print("2. Deposit")
   print("3. Withdraw")
   print("4. Exit")
   choice = input("Enter your choice: ")
    if choice == "1":
      print("Available balance: Rs.", balance)
     elif choice == "2":
      amount = float(input("Enter deposit amount: "))
  if amount > 0:
        balance += amount
        print("Deposit successful.")
        print("Balance:", balance)
      else:
        print("Enter a positive amount.")
```

```
elif choice == "3":
    amount = float(input("Enter withdrawal amount: "))
     if amount <= 0:
        print("Enter a positive amount.")
    elif amount <= balance:
        balance -= amount
        print("Withdrawal successful.")
        print("Balance:", balance)
    else:
        print("Insufficient balance.")
     elif choice == "4":
    print("Thank you for using ATM.")
    break
else:
    print("Invalid choice.")
```

8. Test Cases

Test Case	Operation	Input	Expected Result
1	Check balance	1	Balance ₹5,000
2	Deposit	₹1,000	Balance ₹6,000
3	Withdraw	₹500	Balance ₹4,500 if starting from ₹5,000
4	Withdraw	₹6,000	insufficient balance
5	Exit	4	Program terminates

9.Screenshots



10. Conclusion

The ATM Transaction Simulation demonstrates how Python can be used to model basic banking operations. It applies loops, conditional statements, arithmetic operations, and input validation to process transactions.

CASE STUDY 12: ATTENDANCE RECOVERY COMPUTATION

1. Problem Statement

Develop a Python program to calculate a student's attendance percentage and determine the number of additional classes required to reach a

minimum attendance percentage of 75%, assuming the student attends every future class.

2. Objectives

- To calculate attendance percentages.
- To apply mathematical formulas in Python.
- To use conditional statements.
- To determine the number of classes required to reach a target attendance percentage.

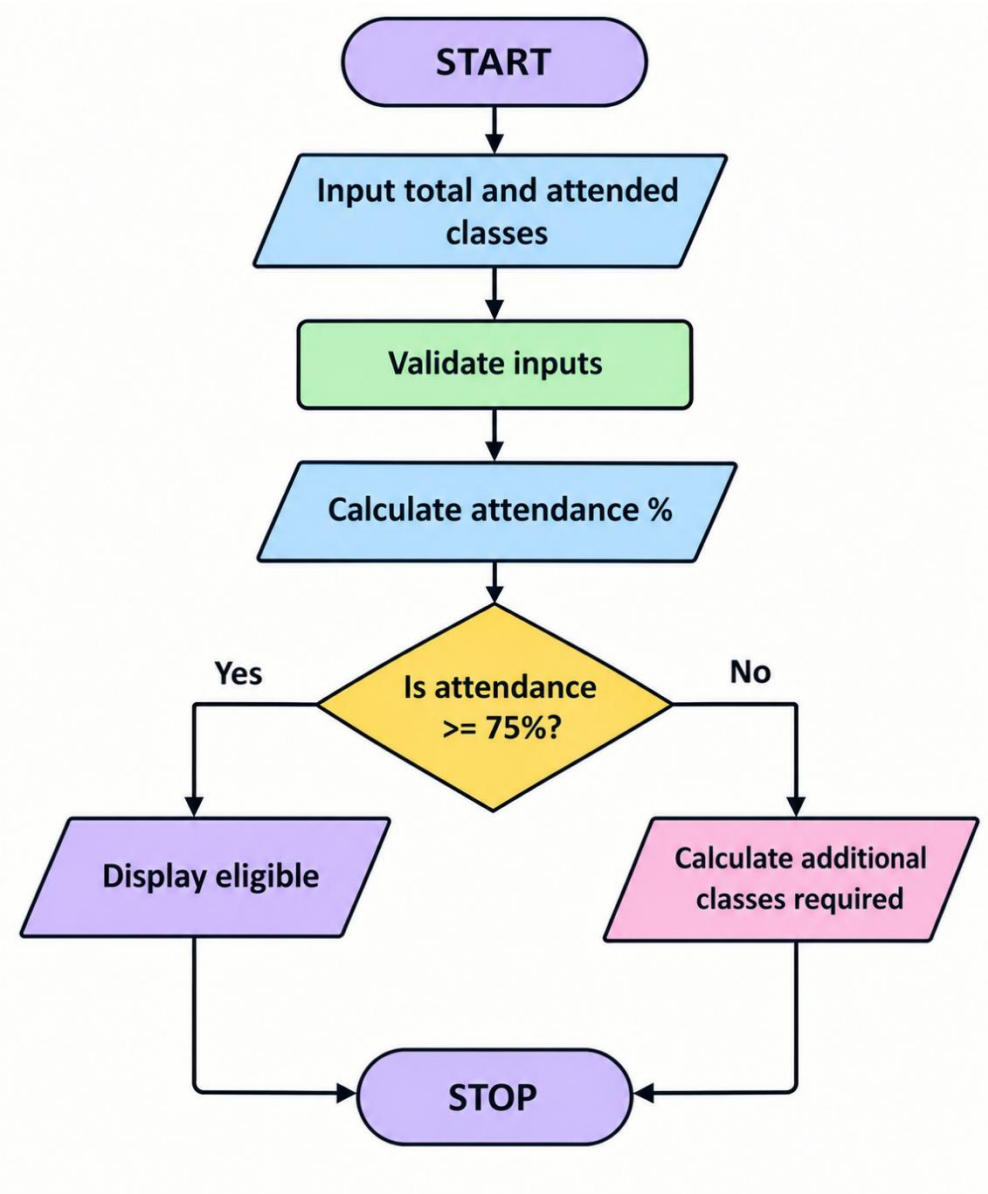
3. Assumptions

- Total classes conducted must be positive.
- Attended classes cannot exceed total classes.
- The minimum required attendance is 75%.
- The student attends all future classes.
- The number of required classes is rounded up to the next whole number.

4. Algorithm

1. Start the program.
2. Input the total number of classes conducted.
3. Input the number of classes attended.
4. Validate the inputs.
5. Calculate attendance percentage.
6. If attendance is at least 75%, display that the requirement is satisfied.
7. Otherwise, calculate the number of additional classes required.
8. Display the result.
9. Stop.

5.Flowchart



6. Pseudocode

BEGIN

INPUT total

INPUT attended

```
    IF total <= 0 OR attended < 0 OR attended > total THEN
      PRINT "Invalid input"
 ELSE
      percentage = attended / total * 100
      PRINT percentage
      IF percentage >= 75 THEN
        PRINT "Attendance requirement satisfied"
      ELSE
        required = CEILING(
          (0.75 * total - attended) / 0.2)
        PRINT required
      END IF
   END IF
END
```

7. Python Source Code

```
import math
print("ATTENDANCE RECOVERY CALCULATOR")
total = int(input("Enter total classes conducted: "))
attended = int(input("Enter classes attended: "))

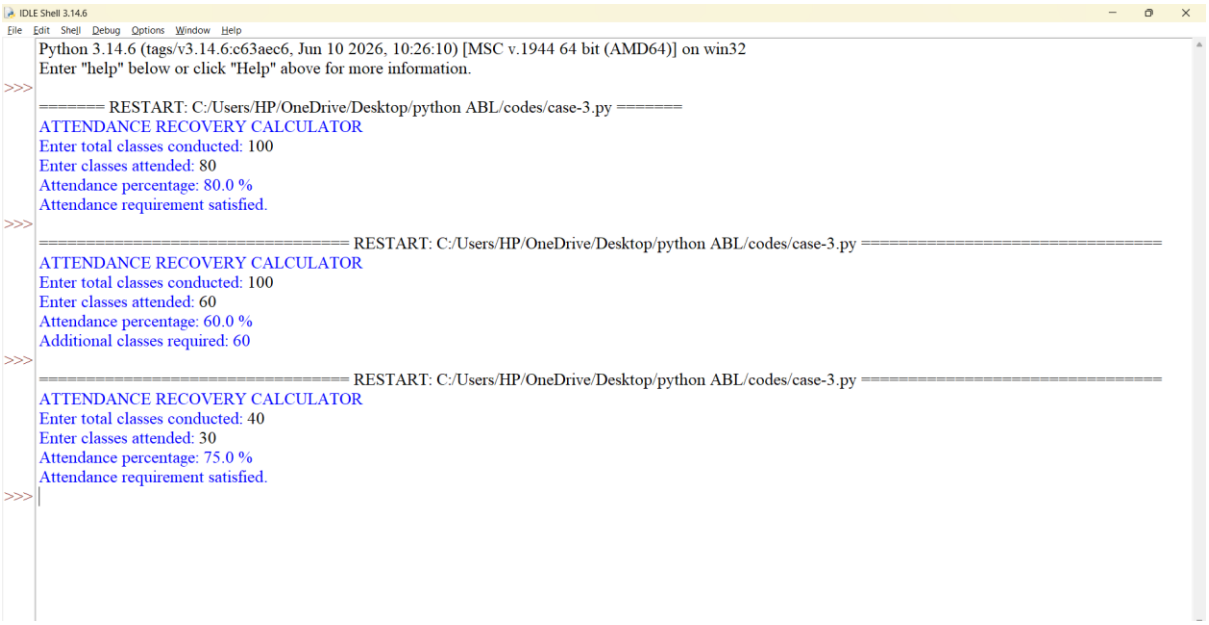
if total <= 0 or attended < 0 or attended > total:
   print("Invalid attendance details.")
else:
   percentage = attended / total * 100
```

```
    print("Attendance percentage:",
          round(percentage, 2), "%")
if percentage >= 75:
        print("Attendance requirement satisfied.")
    else:
        required = math.ceil(
            (0.75 * total - attended) / 0.25)
        print("Additional classes required:", required)
```

8. Test Cases

Test Case	Total Classes	Attended Classes	Expected Attendance	Additional Classes
1	100	80	80%	0
2	100	60	60%	60
3	40	30	75%	0

9.Screenshots



10. Conclusion

The Attendance Recovery Computation program calculates attendance percentages and estimates the number of additional classes required to achieve 75% attendance. It demonstrates the use of mathematical formulas, validation, and conditional statements.

CASE STUDY 17: STUDENT PERFORMANCE TREND ANALYSIS

1. Problem Statement

Develop a Python program to analyze a student's performance across multiple examinations. The program should accept marks from several examinations, calculate the average, identify the highest and lowest marks, and determine whether the student's performance is improving, declining, or remaining stable.

2. Objectives

- To store examination marks in a list.
- To calculate average, maximum, and minimum values.
- To compare examination results.
- To identify performance trends.
- To use loops and conditional statements.

3. Assumptions

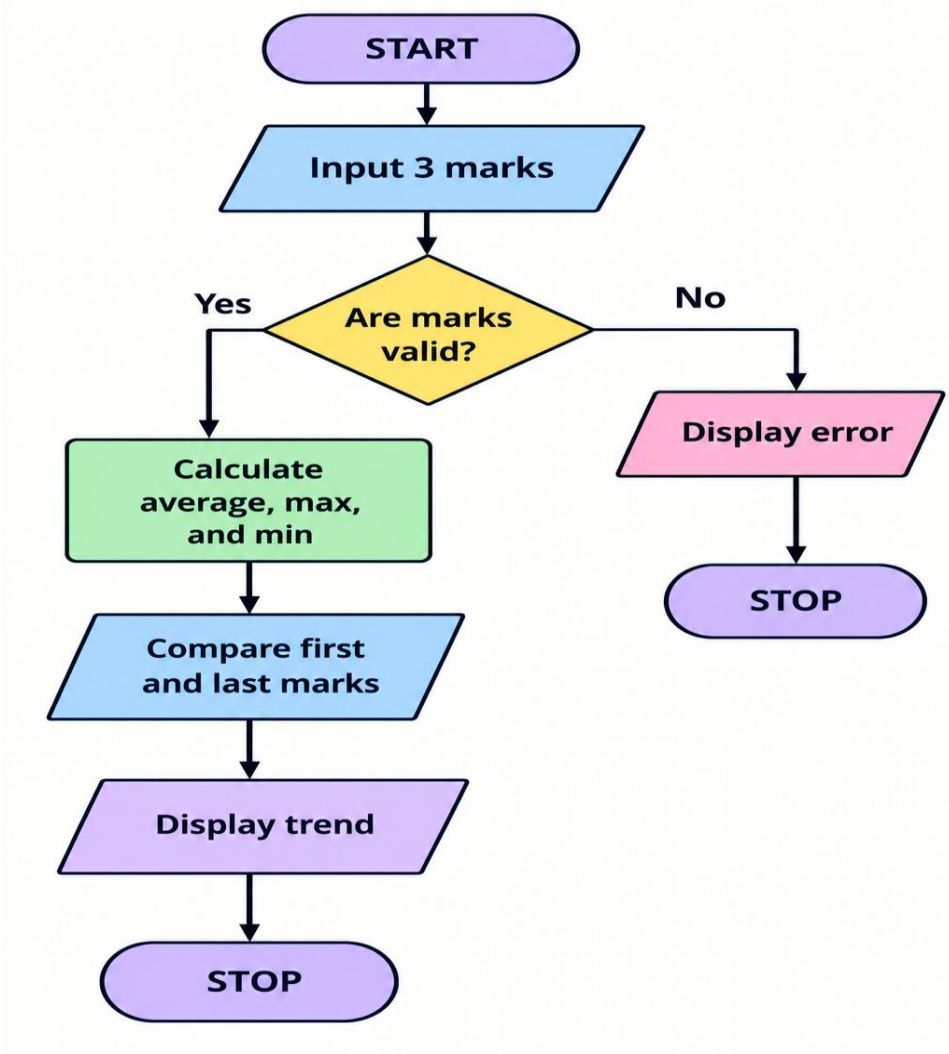
- The student has results from three examinations.
- Marks are entered on a scale of 0 to 100.
- The trend is determined by comparing the first and last examination marks.
- An increase indicates improvement, a decrease indicates decline, and equal marks indicate stable performance.
- Intermediate results are not used to determine the final trend.

4. Algorithm

1. Start the program.
2. Create an empty list to store marks.
3. Accept marks for three examinations.

4. Validate each mark.
5. Calculate the average marks.
6. Find the highest and lowest marks.
7. Compare the first and last marks.
8. Display the performance trend.
9. Stop.

5.Flowcharts



6. Pseudocode

```
BEGIN
  INPUT marks for three examinations
  IF any mark < 0 OR any mark > 100 THEN
    PRINT "Invalid marks"
   ELSE
    average = SUM(marks) / 3
    highest = MAX(marks)
    lowest = MIN(marks)
    IF last mark > first mark THEN
      trend = "Improving"
    ELSE IF last mark < first mark THEN
      trend = "Declining"
    ELSE
      trend = "Stable"
    END IF
    DISPLAY average, highest, lowest, trend
  END IF
END
```

7. Python Source Code

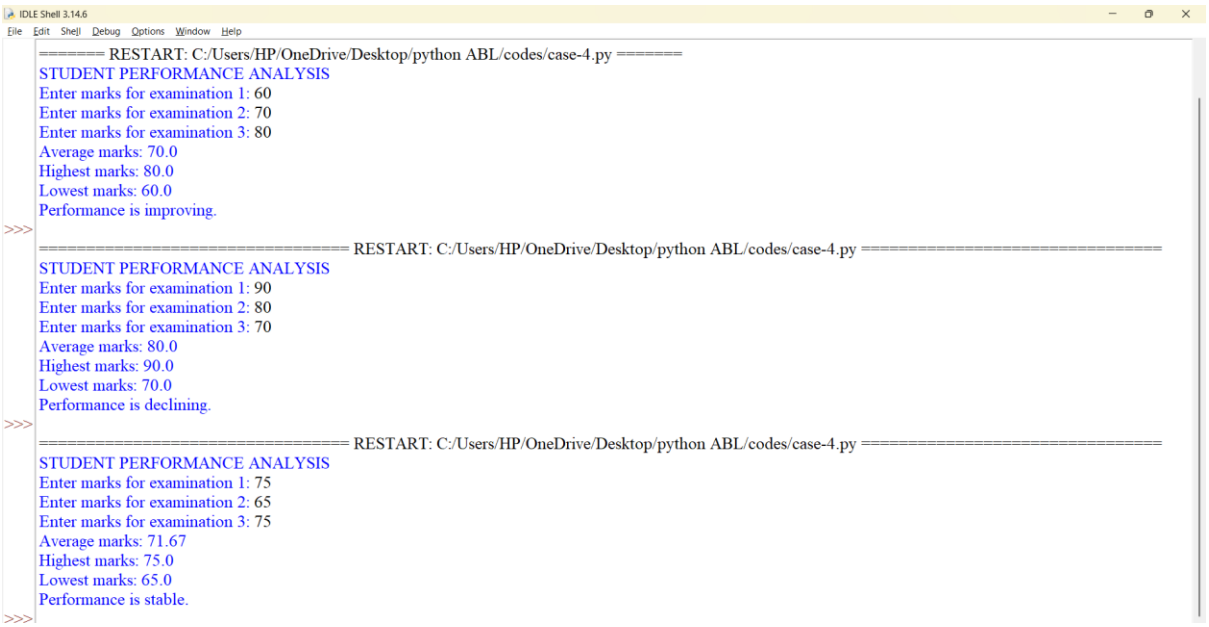
```
print("STUDENT PERFORMANCE ANALYSIS")
marks = []
```

```
for i in range(1, 4):
    mark = float(input(
        f"Enter marks for examination {i}: "))
    if mark < 0 or mark > 100:
        print("Invalid marks. Enter between 0 and 100.")
        break
     marks.append(mark)
if len(marks) == 3:
    average = sum(marks) / 3
    print("Average marks:", round(average, 2))
    print("Highest marks:", max(marks))
    print("Lowest marks:", min(marks))
     if marks[2] > marks[0]:
        print("Performance is improving.")
    elif marks[2] < marks[0]:
        print("Performance is declining.")
    else:
        print("Performance is stable."
```

8. Test Cases

Test Case	Marks	Expected Trend	Average
1	60, 70, 80	Improving	70
2	90, 80, 70	Declining	80
3	75, 65, 75	Stable	71.65

9.Screenshots



10. Conclusion

The Student Performance Trend Analysis program helps evaluate examination results using basic statistical calculations. It demonstrates lists, loops, built-in functions, and conditional statements.

CASE STUDY 18: FUEL CONSUMPTION AND ROUTE COMPARISON

1. Problem Statement

Develop a Python program to compare the fuel consumption and estimated travel cost of two routes. The program should accept the distance of each route, the vehicle's mileage, and the fuel price. It should calculate the fuel required and cost for each route and identify the more economical option.

2. Objectives

- To calculate fuel consumption.
- To estimate travel expenses.
- To compare two routes.
- To apply arithmetic operations.
- To use conditional statements to select the economical route.

3. Assumptions

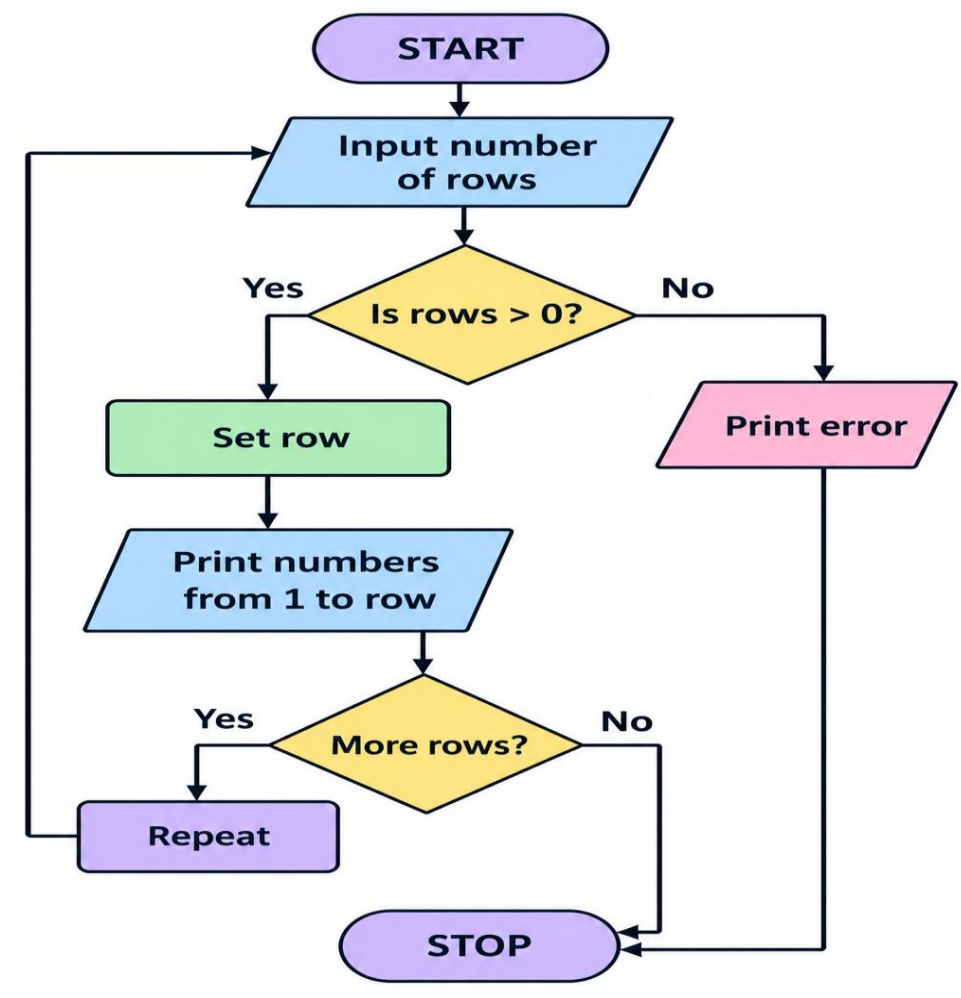
- Distances are measured in kilometres.
- Mileage is measured in kilometres per litre.
- Fuel price is measured in rupees per litre.
- Both routes use the same vehicle and fuel price.
- The route with the lower distance has the lower fuel cost.
- Traffic, tolls, and road conditions are not considered.

4. Algorithm

1. Start the program.
2. Input the distances of Route A and Route B.
3. Input the vehicle mileage.

4. Input the price of fuel per litre.
5. Validate the inputs.
6. Calculate fuel consumption for each route.
7. Calculate the travel cost for each route.
8. Compare the costs.
9. Display the more economical route.
10. Stop.

5. Flowchart



6. Pseudocode

```
BEGIN
   INPUT distanceA
   INPUT distanceB
   INPUT mileage
   INPUT fuelPrice
    IF any input is invalid THEN
      PRINT "Invalid input"
     ELSE
      fuelA = distanceA / mileage
      fuelB = distanceB / mileage
       costA = fuelA * fuelPrice
      costB = fuelB * fuelPrice
       DISPLAY fuelA, fuelB
      DISPLAY costA, costB
      IF costA < costB THEN
         PRINT "Route A is economical"
      ELSE IF costB < costA THEN
         PRINT "Route B is economical"
      ELSE
         PRINT "Both routes cost the same"
      END IF
   END IF
END
```

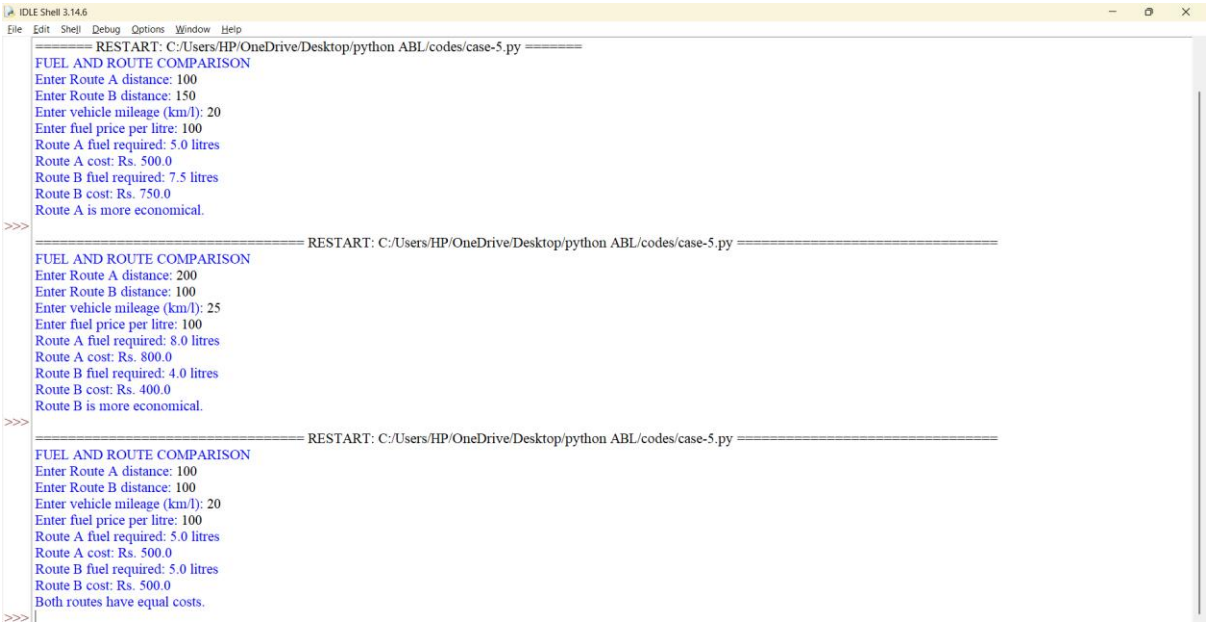
7. Python Source Code

```
print("FUEL AND ROUTE COMPARISON")
distance_a = float(input("Enter Route A distance: "))
distance_b = float(input("Enter Route B distance: "))
mileage = float(input("Enter vehicle mileage (km/l): "))
fuel_price = float(input("Enter fuel price per litre: "))
if (distance_a <= 0 or distance_b <= 0
      or mileage <= 0 or fuel_price <= 0):
   print("Please enter positive values.")
else:
   fuel_a = distance_a / mileage
   fuel_b = distance_b / mileage
    cost_a = fuel_a * fuel_price
   cost_b = fuel_b * fuel_price
   print("Route A fuel required:",
        round(fuel_a, 2), "litres")
   print("Route A cost: Rs.", round(cost_a, 2))
   print("Route B fuel required:",
        round(fuel_b, 2), "litres")
   print("Route B cost: Rs.", round(cost_b, 2))
 if cost_a < cost_b:
      print("Route A is more economical.")
   elif cost_b < cost_a:
      print("Route B is more economical.")
   else:
      print("Both routes have equal costs.")
```

8. Test Cases

Test Case	Route A	Route B	Mileage	Fuel Price	Expected Result
1	100 km	150 km	20 km/l	₹100/l	Route A
2	200 km	100 km	25 km/l	₹100/l	Route B
3	100 km	100 km	20 km/l	₹100/l	Equal costs

9.Screenshots



10. Conclusion

The Fuel Consumption and Route Comparison program calculates estimated fuel requirements and travel costs. It demonstrates how mathematical formulas and conditional statements can help compare travel options.

CASE STUDY 28: REPEATED TRANSACTION DETECTION

1. Problem Statement

Develop a Python program to identify repeated transaction IDs in a list of transaction records. The program should accept multiple transaction IDs, identify duplicates, and display the IDs that appear more than once.

2. Objectives

- To identify duplicate transaction records.
- To understand lists and sets in Python.
- To count the occurrences of transaction IDs.
- To demonstrate a basic method of identifying repeated records.

3. Assumptions

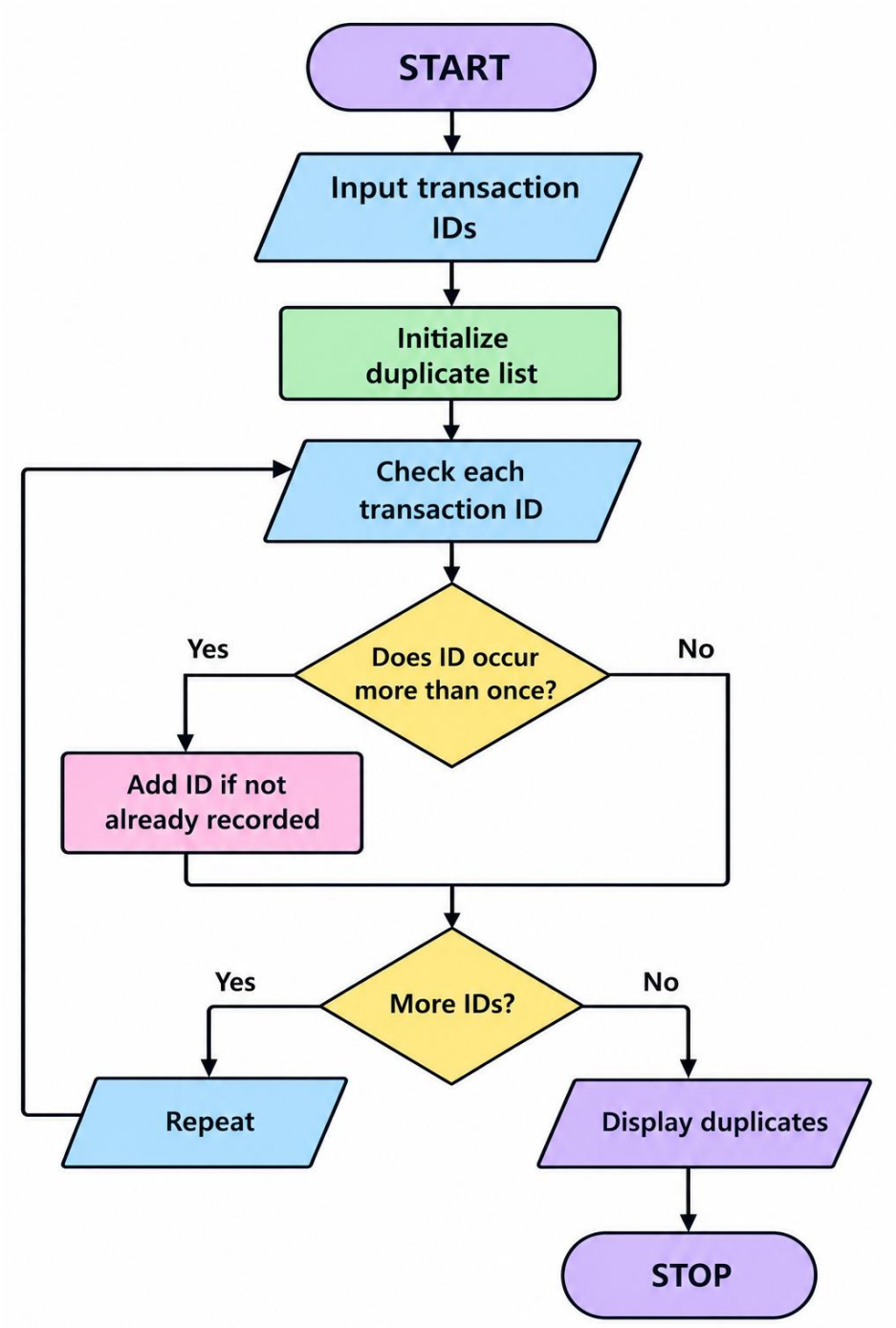
- Transaction IDs are entered as strings.
- IDs are compared exactly as entered.
- Two or more occurrences of an ID indicate a repeated transaction.
- The program identifies duplicate IDs but does not determine whether the transactions are fraudulent.

4. Algorithm

1. Start the program.
2. Input the number of transaction records.
3. Accept the transaction IDs.
4. Create an empty list for duplicate IDs.
5. Check the number of occurrences of each transaction ID.

6. If an ID occurs more than once, add it to the duplicate list if not already present.
7. Display the repeated transaction IDs.
8. If no duplicates exist, display an appropriate message.
9. Stop.

5. Flowchart



6. Pseudo Code

BEGIN

INPUT number of transactions

INPUT transaction IDs

```
  duplicates = empty list

  FOR each ID in transactions
    IF count(ID) > 1 AND ID not in duplicates THEN
      ADD ID to duplicates
    END IF
  END FOR

  IF duplicates is empty THEN
    PRINT "No repeated transactions"
  ELSE
    PRINT duplicates
  END IF
END
```

7. Python Source Code

```
print("REPEATED TRANSACTION DETECTOR")
n = int(input("Enter number of transactions: "))
if n <= 0:
  print("Enter a positive number.")
else:
  transactions = []
    for i in range(n):
     transaction_id = input(
       f"Enter transaction ID {i + 1}: " )
```

```
    transactions.append(transaction_id)

    duplicates = []

    for transaction_id in transactions:
        if (transactions.count(transaction_id) > 1
                and transaction_id not in duplicates):
            duplicates.append(transaction_id)

    if duplicates:
        print("Repeated transaction IDs:")
        for transaction_id in duplicates:
            print(transaction_id)
    else:
        print("No repeated transactions found.")
```

8.Test Cases

Test Case	Transaction IDs	Expected Output
1	T101, T102, T101	T101
2	A1, A2, A3	No duplicates
3	X1, X1, X1	X1 displayed once
4	B1, B2, B1, B2	B1 and B2
5	One transaction: P1	No duplicates

9.Screenshots



10. Conclusion

The Repeated Transaction Detection program identifies duplicate transaction IDs using lists and conditional statements. It demonstrates a simple approach to detecting repeated records in a transaction dataset

CASE STUDY 29: PASSWORD POLICY VALIDATION ENGINE

1. Problem Statement

Develop a Python program to validate a password according to a set of predefined rules. The password must have a minimum length and contain uppercase letters, lowercase letters, digits, and special characters.

2. Objectives

- To validate passwords using Python.
- To understand string operations.
- To use conditional statements.
- To identify uppercase letters, lowercase letters, digits, and special characters.
- To encourage the use of stronger passwords.

3. Assumptions

A password is considered valid when it satisfies all the following conditions:

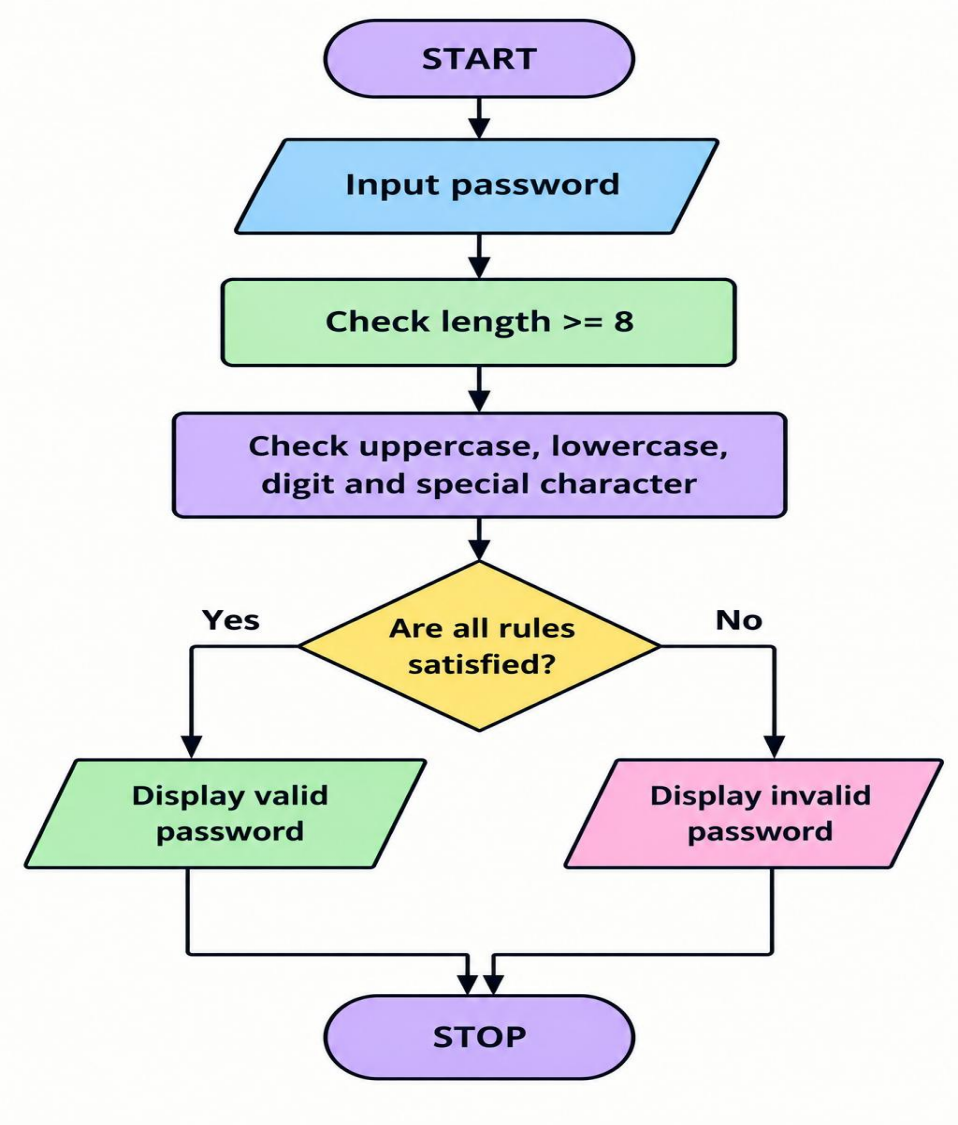
- It contains at least 8 characters.
- It contains at least one uppercase letter.
- It contains at least one lowercase letter.
- It contains at least one digit.
- It contains at least one special character.
- Password strength is determined only by these rules.

4. Algorithm

1. Start the program.

2. Accept a password from the user.
3. Check whether its length is at least 8.
4. Check whether it contains an uppercase letter.
5. Check whether it contains a lowercase letter.
6. Check whether it contains a digit.
7. Check whether it contains a special character.
8. If all conditions are satisfied, display that the password is valid.
9. Otherwise, display the requirements that are not satisfied.
10. Stop.

5. Flowchart



6. Pseudocode

```
BEGIN
   INPUT password
   valid_length = length(password) >= 8
   uppercase = contains uppercase letter
   lowercase = contains lowercase letter
   digit = contains digit
```

```
  special = contains special character
  IF all conditions are TRUE THEN
    PRINT "Valid password"
  ELSE
    PRINT "Invalid password"
    DISPLAY unmet conditions
  END IF
END
```

7. Python Source Code

```
print("PASSWORD VALIDATION ENGINE")
password = input("Enter your password: ")
has_upper = any(ch.isupper() for ch in password)
has_lower = any(ch.islower() for ch in password)
has_digit = any(ch.isdigit() for ch in password)

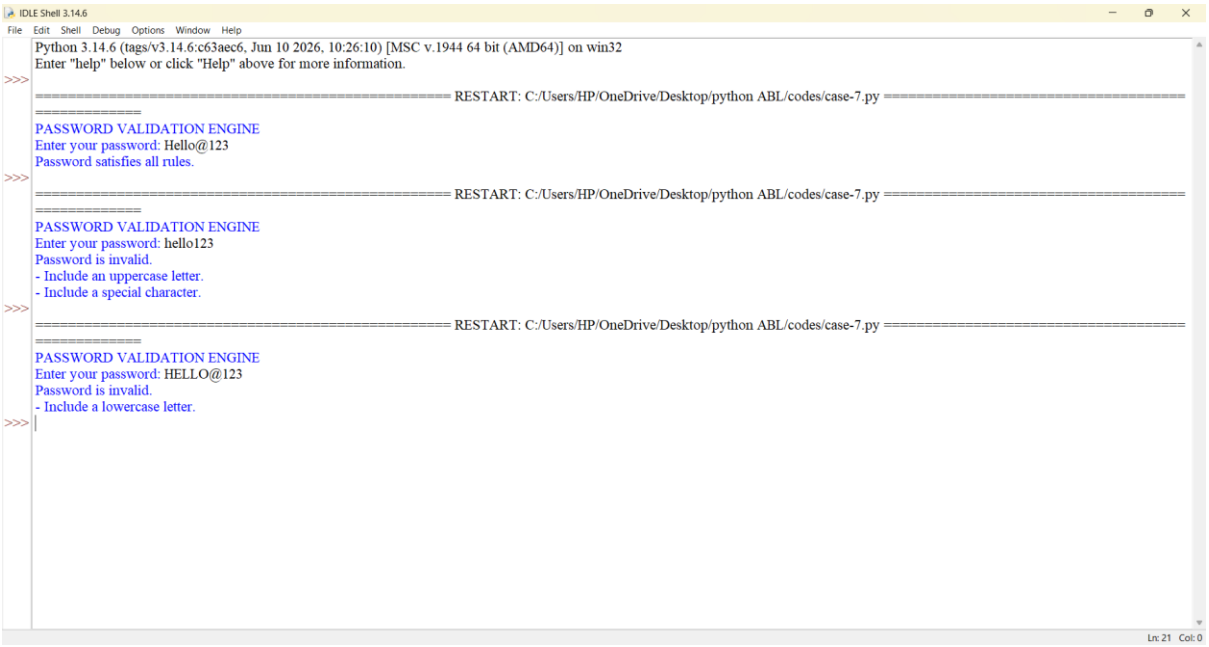
has_special = any(
  not ch.isalnum() and not ch.isspace()
  for ch in password
)
errors = []
if len(password) < 8:
  errors.append("At least 8 characters required.")
```

```
if not has_upper:
    errors.append("Include an uppercase letter.")
if not has_lower:
    errors.append("Include a lowercase letter.")
if not has_digit:
    errors.append("Include a digit.")
if not has_special:
    errors.append("Include a special character.")
if errors:
    print("Password is invalid.")
    for error in errors:
        print("-", error)
else:
    print("Password satisfies all rules.")
```

8. Test Cases

Test Case	Password	Expected Result
1	Hello@123	Valid
2	hello123	Invalid
3	HELLO@123	Invalid

9.screenshots:



10. Conclusion

The Password Policy Validation Engine checks whether a password satisfies predefined complexity requirements. It demonstrates string methods, iteration, Boolean conditions, and input validation. It is an educational example and does not replace a complete password-security system.

CASE STUDY 31: STRING-BASED EXAMINATION ANSWER EVALUATION

1. Problem Statement

Develop a Python program to evaluate a student's examination answers by comparing them with an answer key. The program should calculate the number of correct answers, incorrect answers, and the final score.

2. Objectives

- To compare strings in Python.
- To evaluate examination answers automatically.
- To calculate the final score.
- To use lists and loops.
- To reduce manual evaluation effort.

3. Assumptions

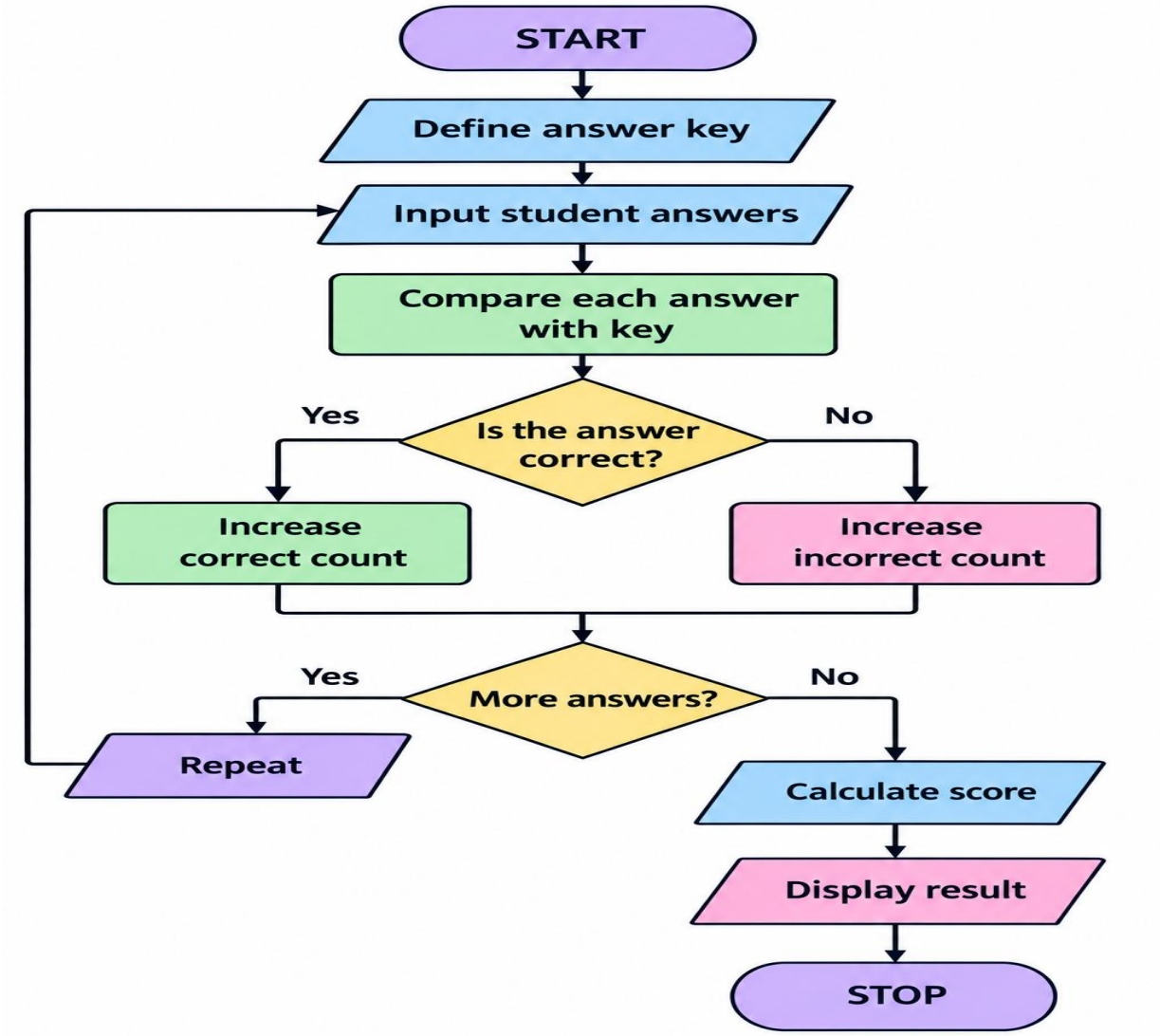
- The examination contains five multiple-choice questions.
- Each question has one correct answer.
- Each correct answer receives one mark.
- Incorrect answers receive zero marks.
- Answers are single letters such as A, B, C, or D.
- Uppercase and lowercase answers are treated equally.
- Negative marking is not applied.

4. Algorithm

1. Start the program.
2. Define the correct answer key.
3. Accept the student's answers.
4. Convert answers to uppercase.
5. Compare each answer with the corresponding answer key.
6. Count the correct and incorrect answers.
7. Calculate the score.
8. Display the result.

9. Stop.

5. Flowchart



6. Pseudocode

BEGIN

answer_key = [A, B, C, D, A]

correct = 0

incorrect = 0

INPUT five student answers

```
  FOR each answer
     CONVERT answer to uppercase
      IF answer matches answer key THEN
        correct = correct + 1
     ELSE
        incorrect = incorrect + 1
     END IF
   END FOR
    score = correct
     DISPLAY correct, incorrect, score
END
```

7. Python Source Code

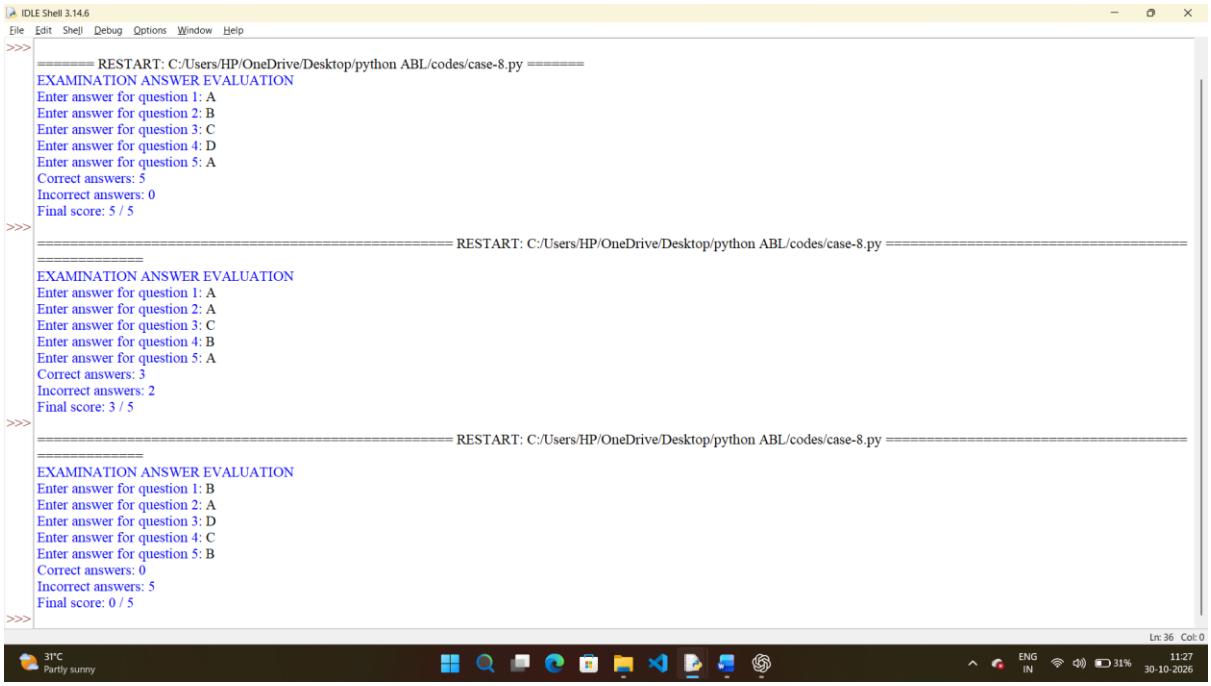
```
print("EXAMINATION ANSWER EVALUATION")
answer_key = ["A", "B", "C", "D", "A"]
student_answers = []
for i in range(5):
   answer = input(
      f"Enter answer for question {i + 1}: "
   ).strip().upper()
  if answer not in ["A", "B", "C", "D"]:
      print("Invalid answer. Use A, B, C, or D.")
      break
  student_answers.append(answer)
```

```
if len(student_answers) == 5:
    correct = 0
    for i in range(5):
        if student_answers[i] == answer_key[i]:
            correct += 1
      incorrect = 5 - correct
    score = correct
    print("Correct answers:", correct)
    print("Incorrect answers:", incorrect)
    print("Final score:", score, "/ 5")
```

8. Test Cases

Test Case	Student Answers	Correct	Incorrect	Score
1	A, B, C, D, A	5	0	5
2	A, A, C, B, A	3	2	3
3	B, A, D, C, B	0	5	

9.Screenshots



10. Conclusion

The String-Based Examination Answer Evaluation program automates the evaluation of multiple-choice answers. It demonstrates string comparison, lists, loops, and counting operations in Python.

CASE STUDY 42: STUDENT MARKS MATRIX ANALYSIS

1. Problem Statement

Develop a Python program to analyze student marks stored in a matrix. Each row represents a student, and each column represents a subject. The program should calculate each student's total and average marks and identify the student with the highest total.

2. Objectives

- To understand two-dimensional lists in Python.
- To store marks in matrix form.
- To calculate student totals and averages.
- To identify the highest-performing student.
- To use nested loops and built-in functions.

3. Assumptions

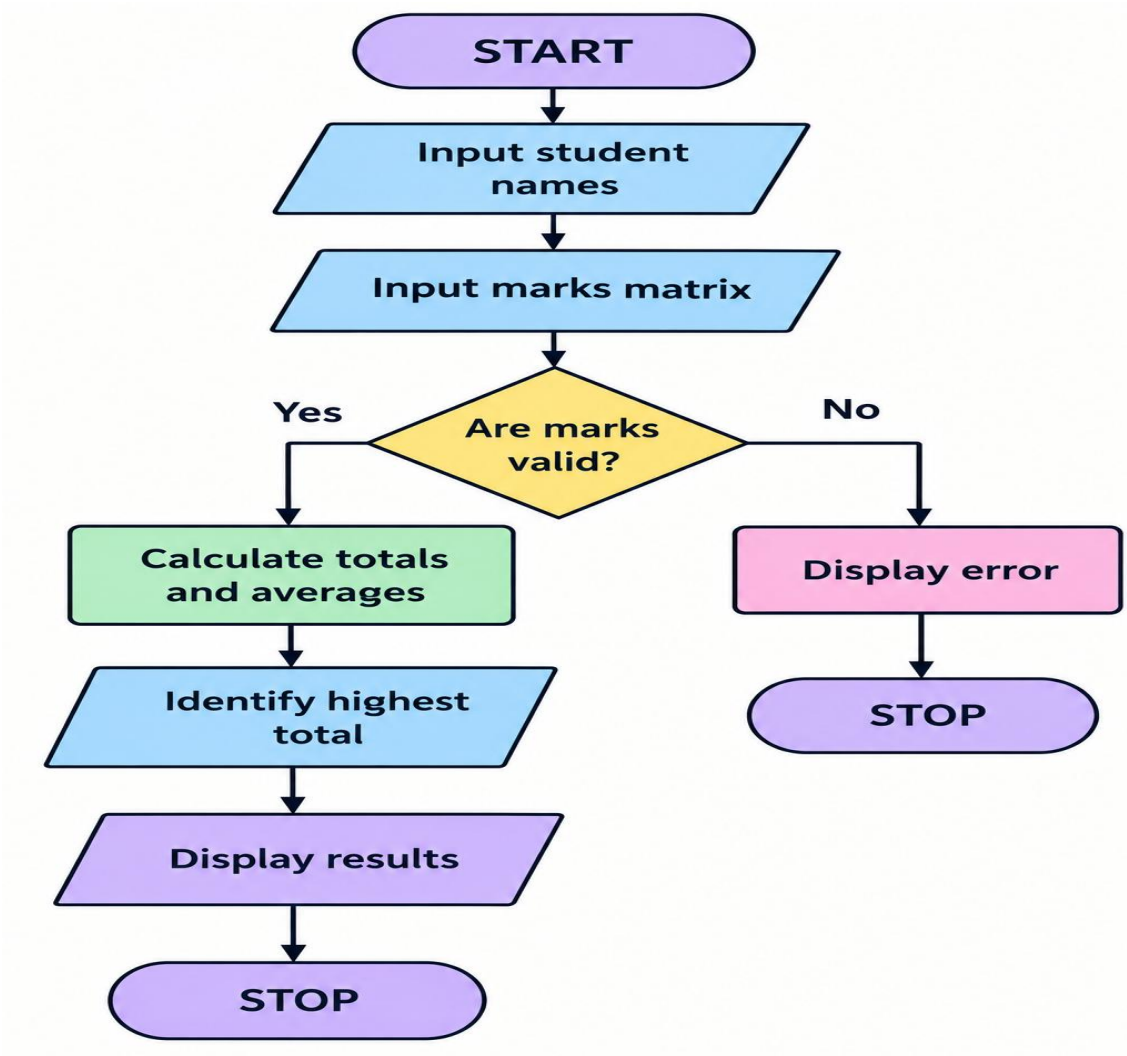
- The program evaluates three students.
- Each student has marks in three subjects.
- Marks range from 0 to 100.
- The matrix contains three rows and three columns.
- Student names are entered separately.
- The student with the highest total is considered the top performer.

4. Algorithm

1. Start the program.
2. Create an empty list for student names.

3. Create an empty matrix for marks.
4. Accept the names of three students.
5. Accept three subject marks for each student.
6. Validate the marks.
7. Calculate each student's total and average.
8. Identify the student with the highest total.
9. Display the results.
10. Stop.

5. Flowchart



6. Pseudocode

```
BEGIN
   students = empty list
   marks = empty matrix
   FOR each of 3 students
      INPUT student name
       INPUT marks for 3 subject
       IF any mark < 0 OR any mark > 100 THEN
         PRINT "Invalid marks"
         STOP
      END IF
      ADD name to students
      ADD marks to matrix
   END FOR
   FOR each student
      total = SUM(student marks)
      average = total / 3
      DISPLAY total and average
   END FOR
    FIND student with highest total
   DISPLAY top student
END
```

7. Python Source Code

```
print("STUDENT MARKS MATRIX ANALYSIS")
students = []
marks = []
for i in range(3):
    name = input(f"Enter student {i + 1} name: ")
    students.append(name)
     row = []
    for j in range(3):
        mark = float(input(
            f"Enter subject {j + 1} marks for {name}: "
        ))
        if mark < 0 or mark > 100:
            print("Invalid marks.")
            break
        row.append(mark)
    if len(row) != 3:
        break
        marks.append(row)
if len(marks) == 3:
    totals = []
    averages = []
```

```
    for row in marks:
        total = sum(row)
        average = total / 3
        totals.append(total)
        averages.append(average)
for i in range(3):
        print("\nStudent:", students[i])
        print("Total:", totals[i])
        print("Average:", round(averages[i], 2))
top_index = totals.index(max(totals))
    print("\nTop performer:", students[top_index])
    print("Highest total:", totals[top_index])
```

8. Test Cases

Test Case	Student	Subject 1	Subject 2	Subject 3	Total
1	Ravi	80	90	70	240
2	Sita	85	95	90	270
3	Aman	70	75	80	225

9.screenshots



10. Conclusion

The Student Marks Matrix Analysis program stores and analyzes student marks using a two-dimensional list. It demonstrates matrix-style data storage, nested loops, arithmetic calculations, and maximum-value identification.

CASE STUDY 53: FOOD WASTE MONITORING SYSTEM

1. Problem Statement

Develop a Python program to monitor food prepared, food consumed, and food wasted. The program should calculate the amount of food wasted and the percentage of food waste. It should also display a message based on the percentage of waste.

2. Objectives

- To monitor food waste.
- To calculate the quantity of wasted food.
- To calculate the percentage of waste.
- To encourage responsible food management.
- To apply arithmetic operations and conditional statements.

3. Assumptions

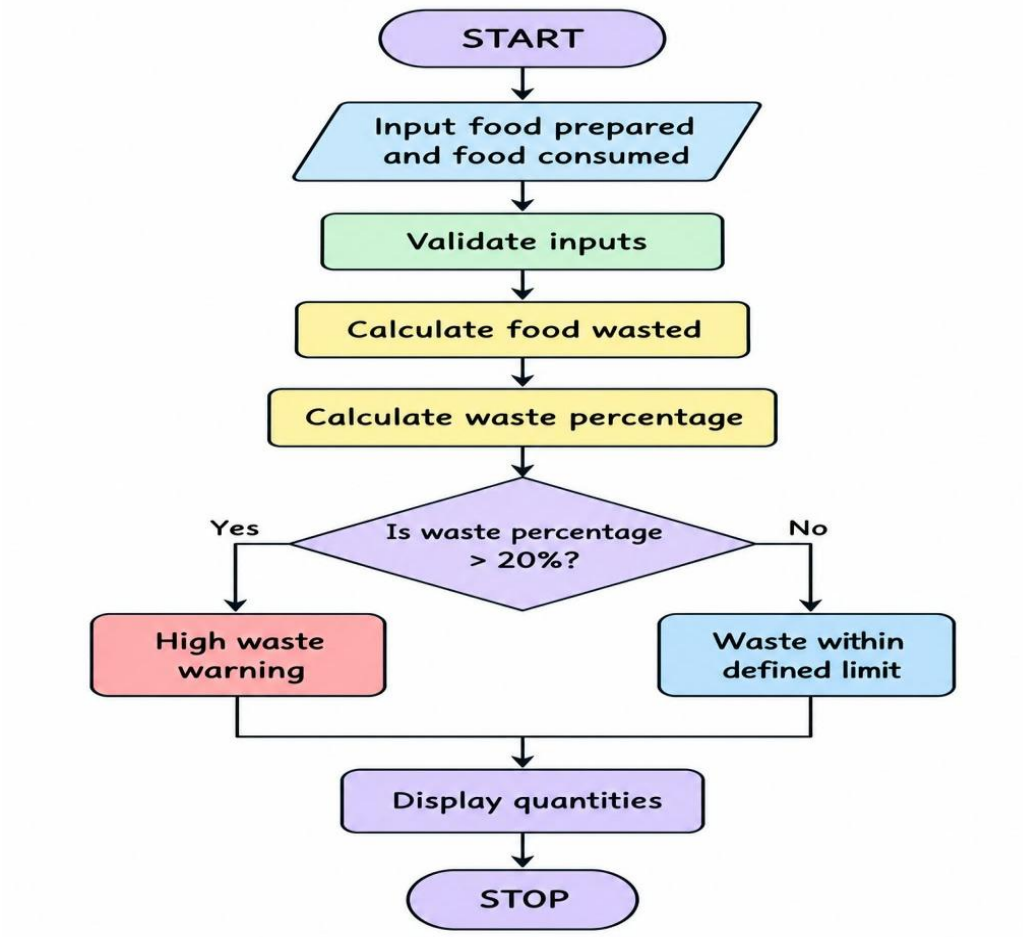
- Food quantities are measured in kilograms.
- Total food prepared must be greater than zero.
- Food consumed cannot exceed food prepared.
- Food wasted equals food prepared minus food consumed.
- A waste percentage greater than 20% is considered high for this educational example.
- The threshold is illustrative and is not an official food-waste standard.

4. Algorithm

1. Start the program.
2. Input the total quantity of food prepared.

3. Input the quantity of food consumed.
4. Validate the quantities.
5. Calculate the wasted food.
6. Calculate the waste percentage.
7. If waste is greater than 20%, display a high-waste warning.
8. Otherwise, display that food waste is within the defined limit.
9. Display all calculated values.
10. Stop.

5. Flowchart



6. Pseudocode

```
BEGIN
   INPUT prepared
   INPUT consumed
   IF prepared <= 0 OR consumed < 0
     OR consumed > prepared THEN
      PRINT "Invalid input"
    ELSE
      wasted = prepared - consumed
      percentage = wasted / prepared * 100
     DISPLAY wasted
      DISPLAY percentage
         IF percentage > 20 THEN
         PRINT "High food waste"
      ELSE
         PRINT "Food waste within defined limit"
      END IF
   END IF
END
```

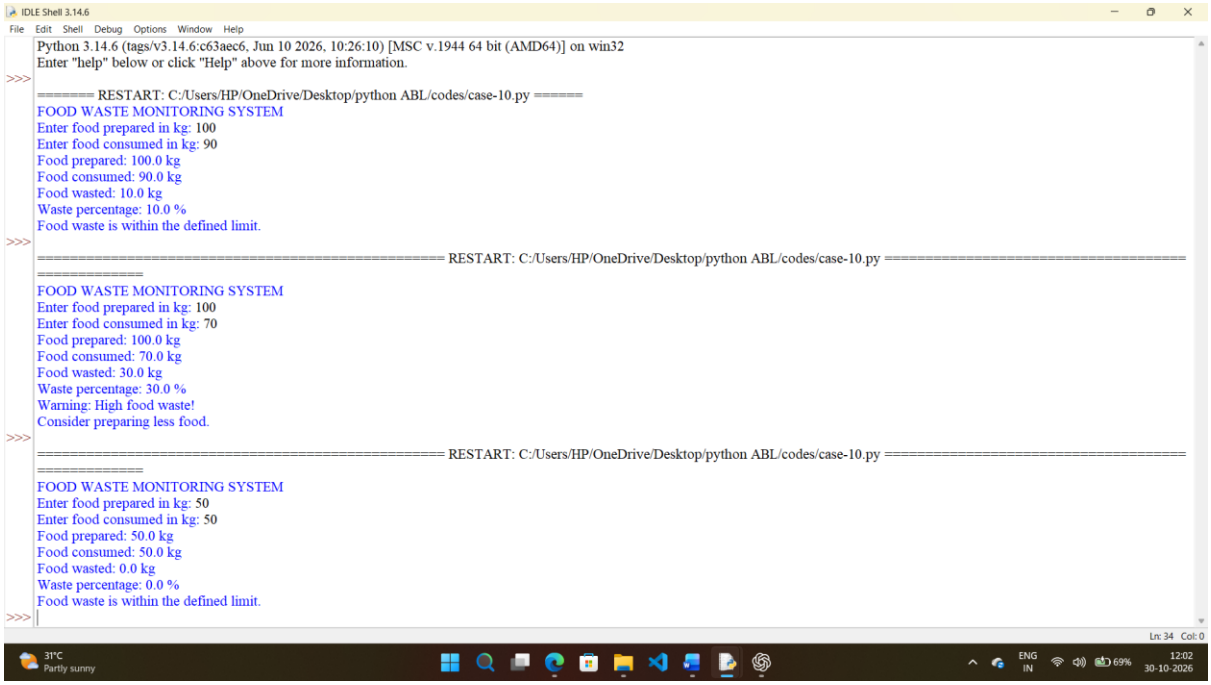
7. Python Source Code

```
print("FOOD WASTE MONITORING SYSTEM")
prepared = float(input(
   "Enter food prepared in kg: "
))
consumed = float(input(
   "Enter food consumed in kg: "
))
if prepared <= 0 or consumed < 0 or consumed > prepared:
   print("Invalid food quantities.")
else:
   wasted = prepared - consumed
   percentage = wasted / prepared * 100
   print("Food prepared:", prepared, "kg")
   print("Food consumed:", consumed, "kg")
   print("Food wasted:", round(wasted, 2), "kg")
   print("Waste percentage:",
         round(percentage, 2), "%")
   if percentage > 20:
      print("Warning: High food waste!")
      print("Consider preparing less food.")
   else:
      print("Food waste is within the defined limit.")
```

8. Test Cases

Test Case	Food Prepared	Food Consumed	Food Wasted	Waste Percentage	Expected Result
1	100 kg	90 kg	10 kg	10%	Within limit
2	100 kg	70 kg	30 kg	30%	High waste
3	50 kg	50 kg	0 kg	0%	Within limit

9.screenshots



10. Conclusion

The Food Waste Monitoring System calculates the amount and percentage of wasted food. It demonstrates the use of mathematical formulas, conditional statements, and input validation while promoting awareness of food conservation.